

SeedHammer II — the pathological wallet

Backing up the constellation's **pathological example** from files: 11 keys, all four Bitcoin timelock kinds, and a hashlock. Host CLIs, the firmware GUI in the emulator, and the plates. No NFC anywhere.

Test material only — never put funds behind these keys.

The three masters are BIP-39's own published test vectors (abandon...about, legal winner...yellow, letter advice...above). They are public by construction.

The wallet

From mnemonic-toolkit's Examples §5, "Custom degrading-miniscript wallet — **the pathological example**". A four-tier vault that deliberately mixes every timelock kind Bitcoin has:

Tier	Spend condition	Timelock	Keys
1	3-of-3 + secret word	after(1000000) — absolute height	@0 @1 @2
2	2-of-3 + secret word	after(1893456000) — absolute time (2030-01-01)	@3 @4 @5
3	both	older(65535) — relative blocks (~455 d)	@6 @7
4	any 1 of 3	older(4255898) — relative time (~365 d)	@8 @9 @10

The secret word is opensesame; the descriptor commits to $H = \text{sha256}(\text{sha256}(\text{word}))$, shared by tiers 1 and 2. Reusing a *hash* across tiers is fine — it is not a key. The 11 keys come from three masters at divergent account indices (48' / 0' / 0' . . 3' / 2'), so no key is reused.

This is the case that actually forces chunking.

The 12-key multi(5,...) policy tried first encodes to 13 bytes and one md1 string — a keyless BIP-388 template does not pay per key. This one carries two 32-byte hash literals and four timelock arguments, and comes to **182 data symbols against a single-string cap of 80**. It is the first policy in this work that genuinely needs a chunk set.

Toolchain

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md --version
md 0.13.0
[exit 0]

$ /scratch/code/shibboleth/mnemonic-key/target/release/mk --version
mk 0.13.0
[exit 0]

$ /scratch/code/shibboleth/mnemonic-secret/target/release/ms --version
ms 0.16.0
[exit 0]

$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me --version
me 0.7.0
[exit 0]

$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me-preview --version
me-preview 0.7.0
[exit 0]
```

The input files

THE POLICY

inputs-pathological/wallet-policy.txt

```
wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad)),multi(3,@0/48'/0'/0'/2'/<0;1>/*,@1/48'/0'/1'/2'/<0;1>/*,@2/48'/0'/2'/2'/<0;1>/*))) ,or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad)),multi(2,@3/48'/0'/3'/2'/<0;1>/*,@4/48'/0'/0'/2'/<0;1>/*,@5/48'/0'/1'/2'/<0;1>/*))) ,or_i(and_v(v:older(65535),multi(2,@6/48'/0'/2'/2'/<0;1>/*,@7/48'/0'/3'/2'/<0;1>/*))) ,and_v(v:older(4255898),multi(1,@8/48'/0'/0'/2'/<0;1>/*,@9/48'/0'/1'/2'/<0;1>/*,@10/48'/0'/2'/2'/<0;1>/*))))))
```

THE THREE MASTERS (SECRET — BIP-39 TEST VECTORS)

inputs-pathological/seeds/master-A.seed

abandon abandon abandon abandon abandon abandon abandon aban
don abandon abandon abandon about

inputs-pathological/seeds/master-B.seed

legal winner thank year wave sausage worth useful legal win
ner thank yellow

inputs-pathological/seeds/master-C.seed

letter advice cage absurd amount doctor acoustic avoid lett
er advice cage above

THE ELEVEN KEYS

inputs-pathological/keys/key-00.xpub

```
# @0 — tier 1 (3-of-3, after height 1000000 + secret word)  
# master A, origin [73c5da0a/48'/0'/0'/2']  
xpub6DKFAXWQ2dHxq2vatrt9qyA3bXYU4ToWQwChbf5XB2mSTexCHZCeKs1  
VZYcPoBd5X8yVcbXFHJR9R8UCVpt82VX1VhR28mCyxUFL4r6KFrF
```

inputs-pathological/keys/key-01.xpub

```
# @1 — tier 1  
# master A, origin [73c5da0a/48'/0'/1'/2']  
xpub6DzhyrnfFYQ1HimDiM388xHnDiRPNdZJFBmxe3Y1WwCHLTMJLfRuh  
RHqnQCPbTj3fGKTuKFLHzwpJkp5Dtc3UtLKZKaVZe1yqMBXd6Vk
```

inputs-pathological/keys/key-02.xpub

```
# @2 — tier 1  
# master A, origin [73c5da0a/48'/0'/2'/2']  
xpub6EGx8sPr9fxPPElrbZazhQwPmXA3Hf5DYKtZbL7c4B5ddzmQktp96U  
aTvecEkoCZysuaj79GMCfZYT1KKk7Ph2M3Kf5g8B82KZ8TZ9SKQR
```

inputs-pathological/keys/key-03.xpub

```
# @3 — tier 2 (2-of-3, after time 1893456000 + secret word)  
# master A, origin [73c5da0a/48'/0'/3'/2']  
xpub6E6Z3Ss5TXJYNjp4U1q3NZ3pCn82i7KXQAKUtnNzLJ3cCdchQeSdFvX  
emizaHUF7wNwRQAB8mPdoZhGLiv49cWpTcnoYJ3Az3E8JKxH9Mq
```

inputs-pathological/keys/key-04.xpub

```
# @4 — tier 2  
# master B, origin [b8688df1/48'/0'/0'/2']  
xpub6FQya7zGhR92kacYsNnjreouvnHJMpxYsUXnW6NJJAJRCKsa26TzDy4  
LdnGhEurrd3d6y1J8PJ7EEMKQp74XTqYvmGJNogYXSKDszyHtF8mX
```

inputs-pathological/keys/key-05.xpub

```
# @5 — tier 2  
# master B, origin [b8688df1/48'/0'/1'/2']  
xpub6EAMBjLn1jiQuajTsNRkZXU1oKnA4WjMNVcz4FRR4QmFKdfHxJVvfRL  
oysWfcc16AMTR4CoMD8UNjvs9JtbsLeuLwpTczgq8zUuERnp8RZF
```

inputs-pathological/keys/key-06.xpub

```
# @6 — tier 3 (both, older 65535 blocks)  
# master B, origin [b8688df1/48'/0'/2'/2']  
xpub6EGceQV5wHrRemjYkYjRWQefTvsNTw1Lr7JUUpTgaawz2P6Gz7So3L  
ktGoyMPshKbCqrgy2RXAmkqBXbqyPe52CnwA1LunyBkNUuMhRt
```

inputs-pathological/keys/key-07.xpub

```
# @7 — tier 3  
# master B, origin [b8688df1/48'/0'/3'/2']  
xpub6ELyn7moeEZ9hJ1HmSm6G5hAfgcCeTa71X9PzYJ39tHNUj93e9MaKb2  
tjAFrMFb94NMTeTG7MW8dwXuhhnPhn5swY6r5dxZH6cyiuPd25AQ5
```

inputs-pathological/keys/key-08.xpub

```
# @8 — tier 4 (any 1 of 3, older 4255898 = ~365d)  
# master C, origin [28645006/48'/0'/2'/2']  
xpub6DnEBNKSJKBYQmsbHs1sP9cNdtU5c9PLFGCjTjmxixc13WB8zNNGQa  
zabOpYFAGW5bV9tMko4uBdxjUKL6dSACx1tEbgEHtgSqyRsekH6
```

inputs-pathological/keys/key-09.xpub

```
# @9 — tier 4  
# master C, origin [28645006/48'/0'/1'/2']  
xpub6F6gx8ZP9R3R3eYsU2PeS5EPJ4jN7Wbt9uwyHNXL0aJxQjNT92FGAfC  
NDjUDRhChWzjfgDuqAZ7Gk9SugPRMa6A8PnzLVvnyEKBW9jHRGRp
```

inputs-pathological/keys/key-10.xpub

```
# @10 — tier 4  
# master C, origin [28645006/48'/0'/2'/2']  
xpub6Dh4twkBRkzxDsau8FdhVY16jARGnPtfDRUK6LbePrGUGok6x8ESvGt  
aygCMLV7beB55YtzkR9dJoic9zYbYy3C7EKc2FGLkKoPT4mQUjPg
```


Host step 1 — it does not fit one string

```
$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-pathological/wallet-policy.txt
wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(3,@0/48'/0'/0'/2'/<0;1>/*,@1/48'/0'/1'/2'/<0;1>/*,@2/48'/0'/2'/2'/<0;1>/*))),or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(2,@3/48'/0'/3'/2'/<0;1>/*,@4/48'/0'/0'/2'/<0;1>/*,@5/48'/0'/1'/2'/<0;1>/*))),or_i(and_v(v:older(65535),multi(2,@6/48'/0'/2'/2'/<0;1>/*,@7/48'/0'/3'/2'/<0;1>/*)),and_v(v:older(4255898),multi(1,@8/48'/0'/0'/2'/<0;1>/*,@9/48'/0'/1'/2'/<0;1>/*,@10/48'/0'/2'/2'/<0;1>/*))))))
[exit 0]

$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode --group-size 0 wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(3,@0/48'/0'/0'/2'/<0;1>/*,@1/48'/0'/1'/2'/<0;1>/*,@2/48'/0'/2'/2'/<0;1>/*))),or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(2,@3/48'/0'/3'/2'/<0;1>/*,@4/48'/0'/0'/2'/<0;1>/*,@5/48'/0'/1'/2'/<0;1>/*))),or_i(and_v(v:older(65535),multi(2,@6/48'/0'/2'/2'/<0;1>/*,@7/48'/0'/3'/2'/<0;1>/*)),and_v(v:older(4255898),multi(1,@8/48'/0'/0'/2'/<0;1>/*,@9/48'/0'/1'/2'/<0;1>/*,@10/48'/0'/2'/2'/<0;1>/*))))))
chunk-set-id: 0x66602
md1fveszps9zjtvvyy5jmplrjttvyy49ykcgfw2fdsssjjtvvyy2fdssj55jmplp9ef9kzz2fdsetu0pjm8rjqqtq
md1fveszpsgs3ef9kzz2j5qqcyefntvqq7sjqfntk5ymnjqjatj0sucqg70h4gdtkemje3rw4fq6srum75s6g2w3
md1fveszps36rc6cckmmg5mngy26xzzqyn9xddhpk7cspxdw6snwgtz4wf7rnqpre774p4wm8gup7frjyxuc3nx
md1fveszps7txyd64y8g0rtrzm0vznwdq3tgcyg69v5e4cqql1l3sgt8fntsypuy6rqpgng12nznqpsv3g40
note: stdout is a keyless descriptor template (no keys)
[exit 0]
```

Host step 2 — so it is chunked, with an explicit origin

Encoded without `--path`, this policy's card carries no origin and md says so: its `wsh(or_i(...))` wrapper has no canonical default derivation path, so md decode only PARTIAL-decodes (exit 4, VERIFY-ME) and me bundle rejects the set outright. Supplying the origin the warning asks for is what makes the rest of the chain work.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode --group-size 0 --force-chunked --fingerprint @0=73c5da0a --fingerprint @1=73c5da0a --fingerprint @2=73c5da0a --fingerprint @3=73c5da0a --fingerprint @4=b8688df1 --fingerprint @5=b8688df1 --fingerprint @6=b8688df1 --fingerprint @7=b8688df1 --fingerprint @8=28645006 --fingerprint @9=28645006 --fingerprint @10=28645006 wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(3,@0/48'/0'/0'/2'/<0;1>/*,@1/48'/0'/1'/2'/<0;1>/*,@2/48'/0'/2'/2'/<0;1>/*))),or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(2,@3/48'/0'/3'/2'/<0;1>/*,@4/48'/0'/0'/2'/<0;1>/*,@5/48'/0'/1'/2'/<0;1>/*))),or_i(and_v(v:older(65535),multi(2,@6/48'/0'/2'/2'/<0;1>/*,@7/48'/0'/3'/2'/<0;1>/*)),and_v(v:older(4255898),multi(1,@8/48'/0'/0'/2'/<0;1>/*,@9/48'/0'/1'/2'/<0;1>/*,@10/48'/0'/2'/2'/<0;1>/*))))))
chunk-set-id: 0xb7e38
md1fkl3czs9zjtvvyy5jmplrjttvyy49ykcgfw2fdsssjjtvvyy2fdssj55jmplp9ef9kvs543f3zlncdz
md1fkl3czsgssjjtvvyy2fdssj54qpxp2v6mqk85yszv6a4pxuusy8unu8xqz8naa2qqzhf8x6au3p6
md1fkl3czss6hdnh9nxa2jr583433dhkpfhxsq45vyyqfx2v6mwrda3qzv6a4pxuusy8fnhjrc4jh6cz
md1fkl3czsa6hyhlpesq3ul02s6hdnh9nxa2jr583433dhkpfhxsq45vzydzk2v6uqqqphmyj94auvedt
md1fkl3cz3rl7xppvaxdwqs8sngvq9zdqecpcpeutks2zu79mq9zw0za5z3h83w6pf9tlcq9n8v2vj5
md1fkl3cz3wrg3hc4hp5gmuts6yd79acdxxlrq5xg5qxj2ry2qr29p9qps94kn53ngu6kxn
note: stdout is a keyless descriptor template (no keys)
[exit 0]
```

What `--path` does and does not say.

It records ONE shared origin — `bip48` and `m/48'/0'/0'/2'` produce a byte-identical chunk set here. These eleven keys actually sit at four account indices (`48'/0'/0'..3'/2'`) across three masters, and each `mk1` card carries its own true `origin_path`. So the `md1`'s shared value is a default the key cards override, and a restore that trusted the descriptor card alone would derive the wrong key for most slots.

That is now pinned, and the pin corrected the claim.

This paragraph used to say the wrong keys were `@3–@10` and that the case was "worth pinning with a restore test". It has been pinned — `restore_test_pathological.py`, run below — and the measured answer is `@1, @2, @3, @5, @6, @7, @9, @10`: eight of eleven, and the range was wrong in *both* directions. It included `@4` and `@8`, which recover fine, and omitted `@1` and `@2`, which do not. Only `@0, @4 and @8` — one per master, each the account-0 key — come back.

Host step 3 — the chunk set decodes back

Three cards, reassembled, give the same 11-key policy back. **That is a transcription check, not a restore test** — it proves the bytes survive the card format and says nothing about whether the origins they carry are the ones the keys actually live at. The restore test is

the next page, and this wallet fails it.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md inspect md1fkl3czs9zjtvyvy5jmplprjttvyvy49ykcgrfw2fdsssjttvyvyw
2fdssj55jmplp9ef9kvs543f3zlncdz md1fkl3czsgssjttvyvyw2fdssj54qqpx2v6mq85yszv6a4pxuusy2unu8xqz8naa2qqzhf8x6au3p6 md1fkl3czss
6hdnh9nza2jr583433dhkpfhxsq45vyyqfx2v6mwrda3qzv6a4pxuusy8fnhjrc4jh6cz md1fkl3czsa6hypesq3ul02s6hdnh9nza2jr583433dhkpfhxsq
45vzydzk2v6uqqqphmyj94auvedt md1fkl3cz3rll7xppvaxdwqqs8sngvq9zdeccpeutks2zu79mg9zw0za5z3h83w6pf9tlcqn9n8v2vj5 md1fkl3cz3wrg
3hc4hp5gmuts6yd79acdzxlrq5xg5qxj2ry2qr29pj9qps94kn53ngu6kxn
template: wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),m
ulti(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*)),or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b96
62375521d0f1ac62dbd829b9a08ad),multi(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*))),or_i(and_v(v:older(65535),multi(2,@6/<0;1>/*,@7/<
0;1>/*)),and_v(v:older(4255898),multi(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))))))
n: 11
wallet-policy-mode: false
md1-encoding-id: b7e387007d6ec40565bd98ace33f4125
wallet-descriptor-template-id: 5b48af35d4321a3ac18b43045e2523cc
origins:
  @0: [73c5da0a/48'/0'/0'/2']
  @1: [73c5da0a/48'/0'/1'/2']
  @2: [73c5da0a/48'/0'/2'/2']
  @3: [73c5da0a/48'/0'/3'/2']
  @4: [b8688df1/48'/0'/0'/2']
  @5: [b8688df1/48'/0'/1'/2']
  @6: [b8688df1/48'/0'/2'/2']
  @7: [b8688df1/48'/0'/3'/2']
  @8: [28645006/48'/0'/0'/2']
... clipped; full text in out/transcript.txt
```

Host step 4 — the key cards bind themselves to the policy

Every mk1 key card carries a 4-byte policy-id stub; mk encode refuses without one. The stub is derived straight from the wallet card with --from-md1, so no hand-copied hex sits between the policy and the eleven cards that claim membership in it:

```
$ /scratch/code/shibboleth/mnemonic-key/target/release/mk encode --xpub xpub6DkFAXWQ2dHxq2vatrt9qyA3bXYU4ToWQwCHbf5XB2mSTexc
HZCeKS1VZYcPoBd5X8yVcbXFHJR9R8UCVpt82VX1VhR28mCyUFL4r6KFrF --origin-fingerprint 73c5da0a --origin-path m/48'/0'/0'/2' --fro
m-md1 md1fkl3czs9zjtvyy5jmpprj2vyy49ykcgfw2fdssj2vyyw2fdssj55jmpp9ef9kvs543f3zlncdz --from-md1 md1fkl3czsgssj2vyyw2fdss
j54qpxp2v6mq85yszv6a4pxuusy2unu8xqz8naa2qqzhf8x6au3p6 --from-md1 md1fkl3czss6hdnh9nxa2jr583433dhkpfhxsq45vyyqfx2v6mwrda3
qzv6a4pxuusy8fnhjr4j6cz --from-md1 md1fkl3czsa6hlypesq3ul02s6hdnh9nxa2jr583433dhkpfhxsq45vzydzk2v6uqqpphmyj94auvedt --fro
m-md1 md1fkl3cz3rll7xppvaxdwqqs8sngvq9zdzqecceutks2zu79mg9zw0za5z3h83w6pf9t1cq9n8v2vj5 --from-md1 md1fkl3cz3wrg3hc4hp5gmutt
s6yd79acd2xlrq5xg5qxj2ry2qr29pj9qps94kn53ngu6kxn --group-size 0
mk1qpd8cwpqqs4k90x4eutks2q5zg3vs7rnefw94m5rru59s2su80aw2q4wgdpapgf14pkhsdytkw15z8lphut2hvvpp5av5muuc0cmfrjw2
mk1qpd8cwp806lhaeh6reknylagmwyjycf8044xtt9flsdlkvt6f6cthy1995lpm5zlrp6yvv6kc36tw
note: stdout is watch-only — public keys only, cannot spend
[exit 0]
```

This did not work when this journey was first walked. mk vendored md-codec 0.34.0 against a primary at 0.42.0, and every chunk was refused with wire-format version mismatch: got 9, expected 4 — so the stub had to be computed by hand and passed as --policy-id-stub. Fixed 2026-08-21 (F-127), and it was two defects stacked: the stale pin, and then mk decoding each --from-md1 value independently, so a four-chunk card was four incomplete sets. Values are now grouped by the 20-bit chunk-set id in their wire header.

THE STUB, CROSS-CHECKED AGAINST THE IDENTITY IT SHOULD BE

The workaround became a check. This card is a keyless template, so the stub must be the top 4 bytes of the key-stable

WalletDescriptorTemplateId — not the key-dependent WalletPolicyId. The journey now reads the stub back out of the card mk just built and fails the build if it disagrees:

```
$ bash -c /scratch/code/shibboleth/descriptor-mnemonic/target/release/md inspect md1fkl3czs9zjtvyy5jmpprj2vyy49ykcgfw2fdss
s2jtvyyw2fdssj55jmpp9ef9kvs543f3zlncdz md1fkl3czsgssj2vyyw2fdssj54qpxp2v6mq85yszv6a4pxuusy2unu8xqz8naa2qqzhf8x6au3p6 md1
fkl3czss6hdnh9nxa2jr583433dhkpfhxsq45vyyqfx2v6mwrda3qzv6a4pxuusy8fnhjr4j6cz md1fkl3czsa6hlypesq3ul02s6hdnh9nxa2jr583433d
hkpfhxsq45vzydzk2v6uqqpphmyj94auvedt md1fkl3cz3rll7xppvaxdwqqs8sngvq9zdzqecceutks2zu79mg9zw0za5z3h83w6pf9t1cq9n8v2vj5 md1fk
l3cz3wrg3hc4hp5gmuts6yd79acd2xlrq5xg5qxj2ry2qr29pj9qps94kn53ngu6kxn 2>/dev/null | grep -E 'wallet-policy-mode|template-id|
wallet-policy-id:'
wallet-policy-mode: false
wallet-descriptor-template-id: 5b48af35d4321a3ac18b43045e2523cc
wallet-policy-id: 024a9921a467b90913bc1569d398a763
[exit 0]

mk --from-md1 embedded 5b48af35 == template-id prefix 5b48af35 (and != policy-id 024a9921)
```

WHICH IDENTITY A KEY CARD INDEXES — SETTLED

Walking this journey turned up a spec/implementation disagreement. SPEC_mk_v0_1.md §3.3 said the stub is "the top 4 bytes of the MD-encoded policy's **WalletPolicyId**", and that is not the one mk uses — it uses the **template-id** for a keyless card. The document originally guessed this was "most likely a rename across versions". **That guess was wrong.** The md-codec pin moved 0.34.0 → 0.42.0 and the behaviour did not change, so the dispatch is deliberate and form-aware: WalletPolicyId for a keyed policy, WalletDescriptorTemplateId for a keyless template. The SPEC was the stale half and was corrected 2026-08-21 (F-128), along with its §5 recovery flow — which, followed literally, would have made a recovery tool reject every card minted from a template.

The consequence is worth carrying: **one wallet has two stubs**, one per form, and membership compares them verbatim.

```
$ bash -c /scratch/code/shibboleth/descriptor-mnemonic/target/release/md inspect md1fkl3czs9zjtvyy5jmpprj2vyy49ykcgfw2fdss
s2jtvyyw2fdssj55jmpp9ef9kvs543f3zlncdz md1fkl3czsgssj2vyyw2fdssj54qpxp2v6mq85yszv6a4pxuusy2unu8xqz8naa2qqzhf8x6au3p6 md1
fkl3czss6hdnh9nxa2jr583433dhkpfhxsq45vyyqfx2v6mwrda3qzv6a4pxuusy8fnhjr4j6cz md1fkl3czsa6hlypesq3ul02s6hdnh9nxa2jr583433d
hkpfhxsq45vzydzk2v6uqqpphmyj94auvedt md1fkl3cz3rll7xppvaxdwqqs8sngvq9zdzqecceutks2zu79mg9zw0za5z3h83w6pf9t1cq9n8v2vj5 md1fk
l3cz3wrg3hc4hp5gmuts6yd79acd2xlrq5xg5qxj2ry2qr29pj9qps94kn53ngu6kxn 2>/dev/null | grep -E 'policy-id|template-id'
wallet-descriptor-template-id: 5b48af35d4321a3ac18b43045e2523cc
wallet-policy-id: 024a9921a467b90913bc1569d398a763
wallet-policy-id-fingerprint: 0x024a9921
[exit 0]
```

Following the binary rather than the sentence, this wallet's stub is **5b48af35**, supplied with --policy-id-stub.

Host step 4 — the eleven key cards

```
$ head -2 /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/keys.txt
[73c5da0a/48'/0'/0'/2']xpub6DkFAXWQ2dHxq2vatrt9qyA3bXYU4ToWQwCHbf5XB2mSTexcHZCeKS1VZYcPoBd5X8yVcbXFHJR9R8UCVpt82VX1VhR28mCyx
UFL4r6KFrF
[73c5da0a/48'/0'/1'/2']xpub6DzhyrnFFYQ1HimDiM388xHnDiRPNdZJF8mmxge3Y1WwCHLTmJLfRuhRHqnQCPbtj3f6KTuKFLHzzwpJkp5Dtc3UtLKZKaVZe
1yqMBXd6Vvk
[exit 0]

$ /scratch/code/shibboleth/mnemonic-key/target/release/mk encode --keys /scratch/code/shibboleth/mnemonic-engrave/design/jou
rneys/out/pathological/keys.txt 12 --from-mdl args --json
note: stdout is watch-only — public keys only, cannot spend
[exit 0]

note: 11 key cards -> 30 mk1 chunks (BIP-48 origins push most cards to 3)
$ /scratch/code/shibboleth/mnemonic-engrave/design/journeys/build_card_index.py /scratch/code/shibboleth/mnemonic-engrave/de
sign/journeys/out/pathological/mk-encode.json /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/key
s-meta.txt /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/card-index.txt
card-index: 30 rows, 11 cards, origins verified
[exit 0]

$ wc -l /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/card-index.txt
30 /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/card-index.txt
[exit 0]

$ wc -l /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/mk-encode-raw.txt
30 /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/mk-encode-raw.txt
[exit 0]

$ /scratch/code/shibboleth/mnemonic-key/target/release/mk decode mk1qpd8cwpqqsq4kj90x4eutks2q5zg3vs7rnefw94m5rru59s2su80aw2q
4wgdppagfl4pkhsdytkw15z8lphut2hvvpp5av5muuc0cmfrjw2 mk1qpd8cwpp806lhaeh6reknylagmwyjycf8044xtt9flsdlkvt6f6cthy1995lpm5z1rp6
yv6kc36tw
xpub:                xpub6DkFAXWQ2dHxq2vatrt9qyA3bXYU4ToWQwCHbf5XB2mSTexcHZCeKS1VZYcPoBd5X8yVcbXFHJR9R8UCVpt82VX1VhR28mCyxUF
L4r6KFrF
origin_fingerprint:  73c5da0a
origin_path:         48'/0'/0'/2'
policy_id_stubs:     5b48af35
chunks:              2 (long)
note: stdout is watch-only — public keys only, cannot spend
[exit 0]
```

Each key splits into 2 or 3 chunks — 30 strings for eleven cards, counted from the transcript's own output rather than assumed. Note the decode: the card carries the origin fingerprint and path, so the origins the descriptor card lacks are present in the bundle.

Host step 5 — the bundle validates, and names every plate

Every set verifies, the manifest emits, and the operator gets the number that matters before touching the machine.

```
$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me bundle --in /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/backup-strings.txt --preview /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/plates --png --manifest /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/manifest.json
me: rendered plate 1 ... (2-25 elided) ... → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/plates/plate-1.png
me: wrote manifest to /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/manifest.json
me: backup needs 37 plates (36 public + msl on device):
  plate 1/37  mdl policy → push via NFC & engrave
  plate 2/37  mdl policy → push via NFC & engrave
  plate 3/37  mdl policy → push via NFC & engrave
  plate 4/37  mdl policy → push via NFC & engrave
  plate 5/37  mdl policy → push via NFC & engrave
  plate 6/37  mdl policy → push via NFC & engrave
  plate 7/37  mk1 [b8688df1/48'/0'/1'/2'] chunk 1/3 → push via NFC & engrave
  plate 8/37  mk1 [b8688df1/48'/0'/1'/2'] chunk 2/3 → push via NFC & engrave
  plate 9/37  mk1 [b8688df1/48'/0'/1'/2'] chunk 3/3 → push via NFC & engrave
  plate 10/37 mk1 [73c5da0a/48'/0'/3'/2'] chunk 1/3 → push via NFC & engrave
  plate 11/37 mk1 [73c5da0a/48'/0'/3'/2'] chunk 2/3 → push via NFC & engrave
  plate 12/37 mk1 [73c5da0a/48'/0'/3'/2'] chunk 3/3 → push via NFC & engrave
  plate 13/37 mk1 [73c5da0a/48'/0'/2'/2'] chunk 1/3 → push via NFC & engrave
  plate 14/37 mk1 [73c5da0a/48'/0'/2'/2'] chunk 2/3 → push via NFC & engrave
  plate 15/37 mk1 [73c5da0a/48'/0'/2'/2'] chunk 3/3 → push via NFC & engrave
  plate 16/37 mk1 [73c5da0a/48'/0'/0'/2'] chunk 1/2 → push via NFC & engrave
  plate 17/37 mk1 [73c5da0a/48'/0'/0'/2'] chunk 2/2 → push via NFC & engrave
  plate 18/37 mk1 [28645006/48'/0'/0'/2'] chunk 1/2 → push via NFC & engrave
  plate 19/37 mk1 [28645006/48'/0'/0'/2'] chunk 2/2 → push via NFC & engrave
  plate 20/37 mk1 [28645006/48'/0'/2'/2'] chunk 1/3 → push via NFC & engrave
  plate 21/37 mk1 [28645006/48'/0'/2'/2'] chunk 2/3 → push via NFC & engrave
  plate 22/37 mk1 [28645006/48'/0'/2'/2'] chunk 3/3 → push via NFC & engrave
  plate 23/37 mk1 [73c5da0a/48'/0'/1'/2'] chunk 1/3 → push via NFC & engrave
  plate 24/37 mk1 [73c5da0a/48'/0'/1'/2'] chunk 2/3 → push via NFC & engrave
  plate 25/37 mk1 [73c5da0a/48'/0'/1'/2'] chunk 3/3 → push via NFC & engrave
  plate 26/37 mk1 [b8688df1/48'/0'/2'/2'] chunk 1/3 → push via NFC & engrave
  plate 27/37 mk1 [b8688df1/48'/0'/2'/2'] chunk 2/3 → push via NFC & engrave
  plate 28/37 mk1 [b8688df1/48'/0'/2'/2'] chunk 3/3 → push via NFC & engrave
  plate 29/37 mk1 [b8688df1/48'/0'/3'/2'] chunk 1/3 → push via NFC & engrave
  plate 30/37 mk1 [b8688df1/48'/0'/3'/2'] chunk 2/3 → push via NFC & engrave
  plate 31/37 mk1 [b8688df1/48'/0'/3'/2'] chunk 3/3 → push via NFC & engrave
  plate 32/37 mk1 [b8688df1/48'/0'/0'/2'] chunk 1/2 → push via NFC & engrave
  plate 33/37 mk1 [b8688df1/48'/0'/0'/2'] chunk 2/2 → push via NFC & engrave
  plate 34/37 mk1 [28645006/48'/0'/1'/2'] chunk 1/3 → push via NFC & engrave
  plate 35/37 mk1 [28645006/48'/0'/1'/2'] chunk 2/3 → push via NFC & engrave
  plate 36/37 mk1 [28645006/48'/0'/1'/2'] chunk 3/3 → push via NFC & engrave
... clipped; full text in out/transcript.txt
```

Host step 6 — the seed, and the refusal that still holds

The addresses — the check this wallet could never do

Until the eleven keys were re-derived at BIP-48's four-level origin, md refused every one of them (expected depth 4 for this script context, got 3) and **no address could be derived for this wallet by any tool**. That is why no earlier version of this document showed one. These are the structural half's counterpart: proof the bytes mean the wallet that was intended.

Item 5: the eleven keys used to sit at BIP-84's three-level single-sig account path, while wsh(<miniscript>) is a MultiSig script context that requires a depth-FOUR xpub. md refused every one of them, so NO ADDRESS could be derived for this wallet by any tool -- which is why no journey ever showed one. Re-derived at m/48'/0'/N'/2', they compose.

This is the FUNCTIONAL half of a round trip: the structural half proves the bytes survived, and this proves they mean the wallet we intended. Compare these against your coordinator before engraving anything.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md address md1fatrz2spzjtvyvyy4qqxpx2v6mqk85ysvz6a4pxuusy2unu8
xqz8naa2r2akwukvgm42gwsr8ztkyupux5za md1fatrz2s2rc6cckmmq5mngy26xzzqyn9xddhpk7cspxdw6snwvgt4wf7rnqpre774p4wmxytldgm8whvtd8 m
d1fatrz2s4evc3h25sapudvvtas2de5z9drq3rg4jnxhqqqrl7xppvaxdwqqs8sngvq9zvd5z6s7pygwfk3 md1fatrz2su9k6zqhswv0j skp2rsal4egz4ep5
859p875x67p5s3wem7sgluxl3d2a3syx3m74wyswrxvcpc54 md1fatrz23zlhah6reknylagmwyjycf8044xtt9fslskvt6f6cthy9yvevrz28llsnr4qy50
0rkxyfe6hw0 md1fatrz23tkxgeal5wa4wy86389z3gq870g9n6rjwuq7cmsyjjgffgkpmqffa5k8pmzr2lxjzdetr8v52fs2cv md1fatrz23n04upy3z0n8ek4a
j5kk9satjt7uv4am7jn7q2pvhex36qvqp5csmqq29aj78uss09z0mledqds8p md1fatrz23m7j8mxp87s87gwfzjsys8njyze0uj4852ajv5x7fyu88u23mx7vw
v6azw8d59kw70a2lmlks0w95 md1fatrz2jx88nfhcc7nza0ynr9p5d3exan7jvup3kx8742q6d662qzwwx3qg6rmwy2r0upegmfdvte0lgjxay md1fatrz2jdk
x6k89kfakaz2az67ajl8hy967je7sks39z80kl7quz6axuvpffejsahhtp2x7gczf3clc26muz md1fatrz2j3w82jq5zap9302ynhp37ggd6z5u4emag0zr8gh9
upnjgr26jfvunvs35jvgdjkvf8nxwprt58vw md1fatrz2jmwxeza6dduqnvtp13vsgcj7vs58sl2hmlrnysxhs57y20d0sn3047m4jnr9cz5e8jumuwp9lg
md1fatrz2n86g9y9evngtkrskaxskvp4qchq5qpv0pez9rarvlx32c8627y2x4unucwuan7ngdj6m8a4jm3a38 md1fatrz2ntm5awv08c2v0vktw5t6qu9g4xkm
v2dywrwkudljyuk9dam643wu0vf30dd15vwq28tgapq6km0dg md1fatrz2n3g59kdh7306na4nhq32cd5w9c2uzlhxryrtzdu2qf455xvp5uc0ktmeyw4h7cajy
9dzt2xm663g md1fatrz2nuquhu772hlj rwlhznxcfd22zx672chhdu5ajtemtchtqljnzwtntna6j74phutvj10dhs8ws22759 md1fatrz25q9z7s576y0hwv6
a92khwlfwmwulc9zrqupz2zll50ju3dcmghtxtfv0y025ltk2r1u3mjg4f4q49y md1fatrz25fnqlk8ycusz0quhfev0a3hqdgghjmsz3ry92d3gv4ejtmu9f0zx
f3clxvtlnnvxgag09mvs08va3 md1fatrz25hgjc5r8x9t5j54jfadzzykqz37sefnc0rhk5zchwu9cfljgk8tg3nlnyr6nv2e7vt5ttahr4hn4 md1fatrz25e
fq2e50d8mg50ts8t8mqquvc90uynctuc4sv9py5lug68qthk65sax8thnpjxetx05qdw2xzn9759 md1fatrz24xrn6wz3pd0pwa4xqjj8e7hte164394h6gme2g4
flvkpvqrj3us7k4x9dcp2a2uz2f0tdhlxnyd7a md1fatrz24w75zn845y9up7y276lt29ywu3usza3aqrp2qarn88u9dtnzuy --chain 0 --count 3
bclqkuknyu6dsm0fq44ccyhzqy9wl3ex2n6ed39zxhx867l9w1h4yhlsejms64
bclq6k2emh2epd57p5qfwswnahtphjlzsuc5x92c7zphr2lnnckzrpzqjdara
bclq97cvjd2wj75hu62w4at6w4ay0mz3janslla3y3zrml4lgyymm38qfh70pa
note: stdout is watch-only – public keys only, cannot spend
[exit 0]
```

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md address md1fatrz2spzjtvyvyy4qqxpx2v6mqk85ysvz6a4pxuusy2unu8
xqz8naa2r2akwukvgm42gwsr8ztkyupux5za md1fatrz2s2rc6cckmmq5mngy26xzzqyn9xddhpk7cspxdw6snwvgt4wf7rnqpre774p4wmxytldgm8whvtd8 m
d1fatrz2s4evc3h25sapudvvtas2de5z9drq3rg4jnxhqqqrl7xppvaxdwqqs8sngvq9zvd5z6s7pygwfk3 md1fatrz2su9k6zqhswv0j skp2rsal4egz4ep5
859p875x67p5s3wem7sgluxl3d2a3syx3m74wyswrxvcpc54 md1fatrz23zlhah6reknylagmwyjycf8044xtt9fslskvt6f6cthy9yvevrz28llsnr4qy50
0rkxyfe6hw0 md1fatrz23tkxgeal5wa4wy86389z3gq870g9n6rjwuq7cmsyjjgffgkpmqffa5k8pmzr2lxjzdetr8v52fs2cv md1fatrz23n04upy3z0n8ek4a
j5kk9satjt7uv4am7jn7q2pvhex36qvqp5csmqq29aj78uss09z0mledqds8p md1fatrz23m7j8mxp87s87gwfzjsys8njyze0uj4852ajv5x7fyu88u23mx7vw
v6azw8d59kw70a2lmlks0w95 md1fatrz2jx88nfhcc7nza0ynr9p5d3exan7jvup3kx8742q6d662qzwwx3qg6rmwy2r0upegmfdvte0lgjxay md1fatrz2jdk
x6k89kfakaz2az67ajl8hy967je7sks39z80kl7quz6axuvpffejsahhtp2x7gczf3clc26muz md1fatrz2j3w82jq5zap9302ynhp37ggd6z5u4emag0zr8gh9
upnjgr26jfvunvs35jvgdjkvf8nxwprt58vw md1fatrz2jmwxeza6dduqnvtp13vsgcj7vs58sl2hmlrnysxhs57y20d0sn3047m4jnr9cz5e8jumuwp9lg
md1fatrz2n86g9y9evngtkrskaxskvp4qchq5qpv0pez9rarvlx32c8627y2x4unucwuan7ngdj6m8a4jm3a38 md1fatrz2ntm5awv08c2v0vktw5t6qu9g4xkm
v2dywrwkudljyuk9dam643wu0vf30dd15vwq28tgapq6km0dg md1fatrz2n3g59kdh7306na4nhq32cd5w9c2uzlhxryrtzdu2qf455xvp5uc0ktmeyw4h7cajy
9dzt2xm663g md1fatrz2nuquhu772hlj rwlhznxcfd22zx672chhdu5ajtemtchtqljnzwtntna6j74phutvj10dhs8ws22759 md1fatrz25q9z7s576y0hwv6
a92khwlfwmwulc9zrqupz2zll50ju3dcmghtxtfv0y025ltk2r1u3mjg4f4q49y md1fatrz25fnqlk8ycusz0quhfev0a3hqdgghjmsz3ry92d3gv4ejtmu9f0zx
f3clxvtlnnvxgag09mvs08va3 md1fatrz25hgjc5r8x9t5j54jfadzzykqz37sefnc0rhk5zchwu9cfljgk8tg3nlnyr6nv2e7vt5ttahr4hn4 md1fatrz25e
fq2e50d8mg50ts8t8mqquvc90uynctuc4sv9py5lug68qthk65sax8thnpjxetx05qdw2xzn9759 md1fatrz24xrn6wz3pd0pwa4xqjj8e7hte164394h6gme2g4
flvkpvqrj3us7k4x9dcp2a2uz2f0tdhlxnyd7a md1fatrz24w75zn845y9up7y276lt29ywu3usza3aqrp2qarn88u9dtnzuy --chain 1 --count 3
bclqxtlrmq23lczrtx8l9x3nsfp3u5h7vt9zjmy94m6srl354ps9amus2q56lx
bclq6keukt5pnwv0mtr593gj8p4yvpvt5gl4hwhzyxzuy066mtj7leaukyk8qt
bclqcy983peqe3n6flckwqf3fvr2g0y2z8hplfxfm8nfecaxzas4ux2jsjwuld
note: stdout is watch-only – public keys only, cannot spend
[exit 0]
```

What this backup will NOT do

Tiers 1–2 cannot be spent from these plates alone. Both need the 32-byte preimage of the sha256 hashlock — the "secret word". **No plate carries it.** Record it separately, with the vault, or those two tiers are decoration.

The tiers do not open in the order they read. Tier 4 (1-of-3) matures at ~365 days and tier 3 (2-of-2) at ~455 — the *weakest* key-set unlocks ~90 days *first*. And each `olDer()` clock runs per-coin from that coin's confirmation, not from the day you engraved.

In master terms this is a 1-of-3 vault with delays, not an 11-key multisig. Three of its four tiers are satisfiable by a single master:

- **A** alone — tier 1 (3-of-3 over @0–@2, all master A), unlocked since block 1,000,000 in 2016 — but only with the secret word;
- **C** alone — tier 4 (1-of-3 over @8–@10), after ~365 days, *no* word needed;
- **B** alone — tier 3 (2-of-2 over @6–@7), after ~455 days.

Only tier 2 needs two masters, and it is the furthest out. So this vault *survives losing any two seed plates* — and *falls to the theft of any one*, given time. That is a deliberate trade if you chose it and a nasty surprise if you did not.

The restore test — what a card-only restore recovers

The decode step is a transcription check. **This is the restore test:** it derives, from each master seed, the key the *descriptor card's own declared origin* would lead a restorer to, and compares it against the key that slot actually holds.

```
$ python3 /scratch/code/shibboleth/mnemonic-engrave/design/journeys/restore_test_pathological.py
```

A) THE OLD CARD — one shared origin, so every master is read at account 0:

```
@0 master 73c5da0a at 48'/0'/0'/2' RECOVERED
@1 master 73c5da0a at 48'/0'/1'/2' WRONG KEY
@2 master 73c5da0a at 48'/0'/2'/2' WRONG KEY
@3 master 73c5da0a at 48'/0'/3'/2' WRONG KEY
@4 master b8688df1 at 48'/0'/0'/2' RECOVERED
@5 master b8688df1 at 48'/0'/1'/2' WRONG KEY
@6 master b8688df1 at 48'/0'/2'/2' WRONG KEY
@7 master b8688df1 at 48'/0'/3'/2' WRONG KEY
@8 master 28645006 at 48'/0'/0'/2' RECOVERED
@9 master 28645006 at 48'/0'/1'/2' WRONG KEY
@10 master 28645006 at 48'/0'/2'/2' WRONG KEY
-> 3 of 11 recovered; wrong: [1, 2, 3, 5, 6, 7, 9, 10]
```

B) THE CARD ENGRAVED NOW — each slot states its own origin:

```
@0 master 73c5da0a at 48'/0'/0'/2' RECOVERED
@1 master 73c5da0a at 48'/0'/1'/2' RECOVERED
@2 master 73c5da0a at 48'/0'/2'/2' RECOVERED
@3 master 73c5da0a at 48'/0'/3'/2' RECOVERED
@4 master b8688df1 at 48'/0'/0'/2' RECOVERED
@5 master b8688df1 at 48'/0'/1'/2' RECOVERED
@6 master b8688df1 at 48'/0'/2'/2' RECOVERED
@7 master b8688df1 at 48'/0'/3'/2' RECOVERED
@8 master 28645006 at 48'/0'/0'/2' RECOVERED
```

... clipped; full text in out/transcript.txt

Three of eleven.

Only @0, @4 and @8 — one account-0 key per master — come back. Every tier of this vault needs signatures this restore cannot produce, so the descriptor card alone is not a backup of this wallet: the mk1 key cards carry the origins that make it one, and they are not optional.

```
$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-pathological/seeds/master-A.seed
```

```
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about
[exit 0]
```

```
$ /scratch/code/shibboleth/mnemonic-secret/target/release/ms encode --phrase abandon abandon abandon abandon abandon abandon
abandon abandon abandon abandon abandon about
```

```
msl0e ntrsqqqqqqqqqqqqqqqqqqqqqqqqqqqqqj9 sxraq 34v7f
```

```
word count: 12
```

```
language: english (BIP-39 checksum valid)
```

```
passphrase: not stored in msl (record separately if used)
```

```
warning: stdout carries private key material (can spend) — redirect or encrypt (e.g. '> file.txt' or '| age -e ...')
```

```
[exit 0]
```

```
$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me --in /dev/fd/63 --hex
```

```
me: refusing to emit msl over NFC: msl is secret seed entropy and must never be transmitted by radio (RF eavesdropping risk). Enter it by hand on the device: New > Input Seed > CODEX32.
```

```
[exit 3]
```

The 36 public plates

6 descriptor chunks, then eleven key cards at two or three chunks each.



plate 1 — md1 policy, chunk 1/6



plate 2 — md1 policy, chunk 2/6



plate 3 — md1 policy, chunk 3/6



plate 4 — md1 policy, chunk 4/6



plate 5 — md1 policy, chunk 5/6



plate 6 — md1 policy, chunk 6/6



plate 7 — @5 [b8688df1/48/0/1/2] chunk 1/3



plate 8 — @5 [b8688df1/48/0/1/2] chunk 2/3



plate 9 — @5 [b8688df1/48/0/1/2] chunk 3/3



plate 10 — @3 [73c5da0a/48/0/3/2] chunk 1/3



plate 11 — @3 [73c5da0a/48/0/3/2] chunk 2/3



plate 12 — @3 [73c5da0a/48/0/3/2] chunk 3/3

The 36 public plates, continued



plate 13 — @2 [73c5da0a/48/0/2/2] chunk 1/3



plate 14 — @2 [73c5da0a/48/0/2/2] chunk 2/3



plate 15 — @2 [73c5da0a/48/0/2/2] chunk 3/3



plate 16 — @0 [73c5da0a/48/0/0/2] chunk 1/2



plate 17 — @0 [73c5da0a/48/0/0/2] chunk 2/2



plate 18 — @8 [28645006/48/0/0/2] chunk 1/2



plate 19 — @8 [28645006/48/0/0/2] chunk 2/2



plate 20 — @10 [28645006/48/0/2/2] chunk 1/3



plate 21 — @10 [28645006/48/0/2/2] chunk 3/3



plate 22 — @10 [28645006/48/0/2/2] chunk 3/3



plate 23 — @1 [73c5da0a/48/0/1/2] chunk 1/3



plate 24 — @1 [73c5da0a/48/0/1/2] chunk 2/3



plate 25 — @1 [73c5da0a/48/0/1/2] chunk 3/3



plate 26 — @6 [b8688df1/48/0/2/2] chunk 1/3



plate 27 — @6 [b8688df1/48/0/2/2] chunk 2/3



plate 28 — @6 [b8688df1/48/0/2/2] chunk 3/3



plate 29 — @7 [b8688df1/48/0/3/2] chunk 1/3



plate 30 — @7 [b8688df1/48/0/3/2] chunk 2/3



plate 31 — @7 [b8688df1/48/0/3/2] chunk 3/3



plate 32 — @4 [b8688df1/48/0/0/2] chunk 1/2



plate 33 — @4 [b8688df1/48/0/0/2] chunk 2/2



plate 34 — @9 [28645006/48/0/1/2] chunk 1/3



plate 35 — @9 [28645006/48/0/1/2] chunk 2/3



plate 36 — @9 [28645006/48/0/1/2] chunk 3/3

Plus three seed plates that are not here.

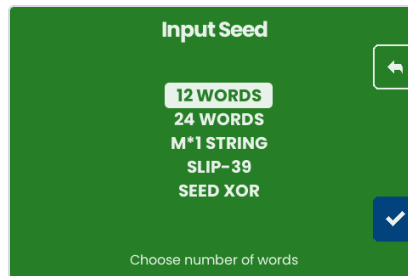
Each master is engraved from words typed on the machine. No host tool will render, transmit or preview them — so the real total is **37 plates**.

On the machine — the seed, typed

No NFC in this journey. The seed goes in the way the machine's own checklist says it must: by hand, on the device.



Boot: Backup Wallet



Input Seed — 12 words



Word 1 of 12



Three letters is enough: "1 match", auto-completed



All 12 accepted — the BIP-39 checksum holds



Optional BIP-39 passphrase, skipped

On the machine — the seed plate

Engrave Plate

Insert a blank plate and close the lock.
Hold button to start the engraving process. The process is loud, use hearing protection.

↓

Insert a blank plate; hold to start

Engrave Plate

13:39

Engraving plate

Cutting

Engrave Plate

5:05

Engraving plate

Still cutting — a seed plate is the long one

The plate overlay, live

The whole planned layout is drawn in **grey** the moment the engrave begins, from the same PlanEngraving call the firmware makes. What the driver has actually stepped is filled in **black**, decoded from the real step stream. The red circle is the head. The title is the master fingerprint 73C5DA0A — the same one the descriptor's origins name.

73C5DA0A

1 ABANDON
2 ABANDON
3 ABANDON
4 ABANDON
5 ABANDON
6 ABANDON
7 ABANDON
8 ABANDON
9 ABANDON
10 ABANDON
11 ABANDON
12 ABOUT

head 25.9, 41.8 mm — 18% of the cut (34040352 steps)

73C5DA0A

1 ABANDON
2 ABANDON
3 ABANDON
4 ABANDON
5 ABANDON
6 ABANDON
7 ABANDON
8 ABANDON
9 ABANDON
10 ABANDON
11 ABANDON
12 ABOUT

head 24.1, 64.4 mm — 41% of the cut (77422644 steps)

73C5DA0A

1 ABANDON
2 ABANDON
3 ABANDON
4 ABANDON
5 ABANDON
6 ABANDON
7 ABANDON
8 ABANDON
9 ABANDON
10 ABANDON
11 ABANDON
12 ABOUT

head 56.7, 44.0 mm — 71% of the cut (134197290 steps)

73C5DA0A

1 ABANDON
2 ABANDON
3 ABANDON
4 ABANDON
5 ABANDON
6 ABANDON
7 ABANDON
8 ABANDON
9 ABANDON
10 ABANDON
11 ABANDON
12 ABOUT

head 59.4, 32.0 mm — 93% of the cut (177152436 steps)

What this run turned up

1. `mk encode --from-md1` could not read a chunked md1. FIXED 2026-08-21 (F-127).

Every chunk was refused with `wire-format version mismatch: got 9, expected 4, exit 2` — and a stub is mandatory, so for any policy large enough to need chunking the documented route to a key card did not work. This document's original workaround was to read the identity out of `md inspect` and pass `--policy-id-stub` by hand.

It was **two** defects stacked, the second invisible until the first was fixed. `mk` vendored `md-codec 0.34.0` against a primary at `0.42.0` — version 9 is the chunked wire form its copy predated. Once bumped, the error only *changed*, to `chunk set incomplete: got 1 chunks, expected 4`, because each `--from-md1` value was decoded independently: a four-chunk card was four incomplete sets. Values are now grouped by the 20-bit chunk-set id in their wire header.

Scale, measured afterwards: a keyed wallet policy is 246 data symbols against a single-string cap of 80, so **every** keyed card in the constellation is chunked. Not a large-wallet corner — the feature was absent for all of them and only ever worked on templates short enough to fit one string.

2. The spec and the binary disagreed about which identity the stub indexes. FIXED 2026-08-21 (F-128) — in the SPEC, not the code.

`SPEC_mk_v0_1.md §3.3` said the stub is the top 4 bytes of the **WalletPolicyId**, and `md` prints a field of exactly that name. Measured, `mk` embeds the top 4 bytes of the **wallet-descriptor-template-id** instead — 726a6663 where the `wallet-policy-id` began `f05e8a1c`.

This document originally guessed that was "probably a rename that landed in `md-codec` after 0.34.0". **The guess was wrong.** The pin moved `0.34.0` → `0.42.0` and the behaviour did not change, establishing the dispatch as deliberate and form-aware: `WalletPolicyId` for a keyed policy, `WalletDescriptorTemplateId` for a keyless template. The SPEC was the stale half. Its §5 recovery flow mattered more than §3.3 — followed literally it would have made a recovery tool reject every card minted from a template, and present it as "none of my cards belong to this wallet".

The consequence the spec now states: **one wallet has two stubs**, one per form, and membership compares them verbatim.

3. `--path` is required here, and it flattens.

Without it the descriptor card carries no origin, `md decode PARTIAL`-decodes (exit 4) and `me bundle` refuses the set. With it, everything validates — but it records a single shared origin while these eleven keys sit at four account indices. The true paths survive on the `mk1` cards; the descriptor card alone restores only **3 of 11** slots correctly — `@0`, `@4` and `@8`, one account-0 key per master. Measured by the restore test above, which also corrected this sentence's earlier claim of "`@3–@10`". Which source wins is a *design* question rather than a defect, and it is now pinned rather than argued.

4. `md encode` has no per-key origin flag.

`--key` takes a bare xpub and rejects an origin-annotated one (`base58check decode`), and `--fingerprint` carries no path. So a divergent-origin wallet cannot state its origins in the descriptor card at all — only the flattened form of finding 3 is expressible.

5. Carried over from the first journey:

presenting an NFC tag to a gathering flow freezes the emulator (filed as F-126). This journey avoids NFC entirely, so it is not exercised here.

Corrected from the first draft of this document.

It reported `me bundle`'s refusal as a tool defect and rendered the plates by calling the sidecar directly. That was wrong: the refusal was the consequence of omitting `--path`, which `md` had warned about at encode time. The plates and checklist here come from `me bundle --preview`.

Reproducing this document

```
cd <this directory>
bash transcript_pathological.sh > transcript_pathological.txt 2>&1 # every CLI block, refusals included
python3 capture_pathological.py # the 13 screenshots, from the emulator
python3 build_pdf_pathological.py # this document, as a PDF
```

Emulator frames were captured by POSTing `canvas.toDataURL()` to a local receiver, so each screenshot is the device framebuffer exactly. Plate overlays are the page's own SVG, rasterised in the browser that drew them. Both are produced by `capture_pathological.py`, which drives `cmd/emu/shots_pathological.js` — the capture used to be console code in a session that no longer exists, which is why this document could not be rebuilt (F-210).